

B28: SQL SuperHero Program : Session 27

1 message

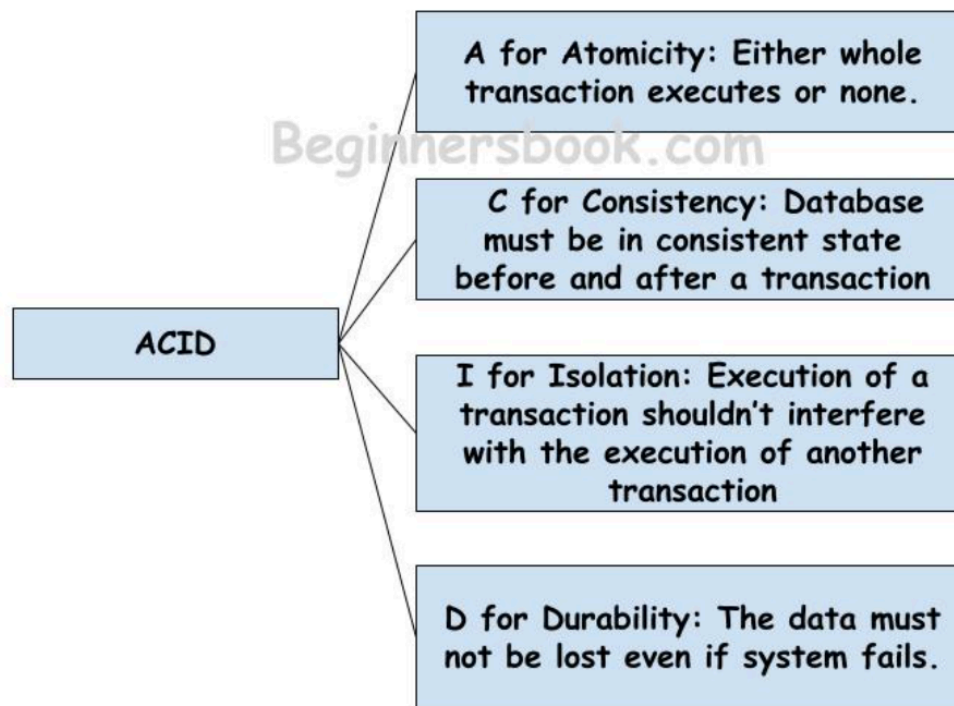
Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

31 March 2025 at 18:50

ACID Properties :

A transaction is a single logical unit of work which accesses and possibly modifies the contents of a database. Transactions access data using read and write operations.

In order to maintain consistency in a database, before and after the transaction, certain properties are followed. These are called ACID properties.



Atomicity

By this, we mean that either the entire transaction takes place at once or doesn't happen at all. There is no midway i.e. transactions do not occur partially.

—**Abort**: If a transaction aborts, changes made to database are not visible.

—**Commit**: If a transaction commits, changes made are visible.

Atomicity is also known as the 'All or nothing rule'.

Consistency

This means that integrity constraints must be maintained so that the database is consistent before and after the transaction.

Isolation

This property ensures that multiple transactions can occur concurrently without leading to the inconsistency of database state. Transactions occur independently without interference.

Durability:

This property ensures that once the transaction has completed execution, the updates and modifications to the database are stored in and written to disk and they persist even if a system failure occurs.

Always explain this question with ATM example as we discussed in class.

1. try to withdraw a \$1000 but my account just have \$500, ATM machine never give me \$500 that my account has just \$500 , take this. NO.

it will give error "insufficient balance"

so it means, what ever transaction asked, entire transaction takes place or nothing. this is atomicity

2. same ATM machine, have \$500 in my account. either I will check in Newyork, london, new delhi, california. all over the globe, my atm is \$500. If I withdraw \$100 in washington DC, the whole world ATM start showing \$400. this is consistency. so database is consistent before and after the transaction

3. isolation : all transactions all independent. same atm example. if my first transaction gives an error, i am still able to do other transactions, like account balance, mini statement. all are independent.

4. Durability : same ATM example, even if system failure then still our transaction will be in place.

The ACID properties ensure the integrity and the consistency of the data in the database, ensuring that the data does not become corrupt as a result of some failure. The databases that apply the ACID properties will ensure that only transactions that were completely successful will be processed, and if some failure happens before a transaction is completed, the data will not be changed.

=====

PL/SQL

PL/SQL is a powerful, yet straightforward database programming language. PL/SQL is a block-structured language; familiarity with blocks is critical to writing good code. PL/SQL which is a block-structured language, this means that the PL/SQL programs are divided and written in logical blocks of code. Each block consists of three sub-parts –

1 Declarations

This section starts with the keyword DECLARE. It is an optional section and defines all variables, cursors, subprograms, and other elements to be used in the program.

2 Executable Commands

This section is enclosed between the keywords BEGIN and END and it is a mandatory section. It consists of the executable PL/SQL statements of the program. It should have at least one executable line of code, which may be just a NULL command to indicate that nothing should be executed.

3 Exception Handling

This section starts with the keyword EXCEPTION. This optional section contains exception(s) that handle errors in the program.

Every PL/SQL statement ends with a semicolon (;). PL/SQL blocks can be nested within other PL/SQL blocks using BEGIN and END.

Following is the basic structure of a PL/SQL block –

```
DECLARE
  <declarations section>
BEGIN
  <executable command(s)>
EXCEPTION
  <exception handling>
END;
```

- The code block is terminated by the End keyword. This is the only keyword within the
- construct that is followed by a semi-colon (;). The only required section is the executable

- section. This means the code block must have the Begin and End keywords.

```
=====
=====
```

Assume that you need to issue multiple SELECT statements. Each SELECT statement is a request against the database and is sent to the Oracle server. The results of each SELECT statement are sent back to the client. Each time a SELECT statement is executed, network traffic is generated. Hence, multiple SELECT statements result in multiple round-trip transmissions, adding significantly to the network traffic.

When these SELECT statements are combined into a PL/SQL program, they are sent to the server as a single unit. The SELECT statements in this PL/SQL program are executed at the server. The server sends the results of these SELECT statements back to the client, also as a single unit. Therefore, a PL/SQL program encompassing multiple SELECT statements can be executed at the server and have the results returned to the client in one round trip. This obviously is a more efficient process than having each SELECT statement executed independently

Why is it more efficient to combine SQL statements into PL/SQL blocks?

It is more efficient to use SQL statements within PL/SQL blocks because network traffic can be decreased significantly, and an application becomes more efficient as well.

When a SQL statement is issued on the client computer, the request is made to the database on the server, and the result set is sent back to the client. As a result, a single SQL statement causes two trips on the network.

If multiple SELECT statements are issued, the network traffic can quickly increase significantly. For example, four SELECT statements cause eight network trips. If these statements are part of the PL/SQL block, still only two network trips are made, as in the case of a single SELECT statement

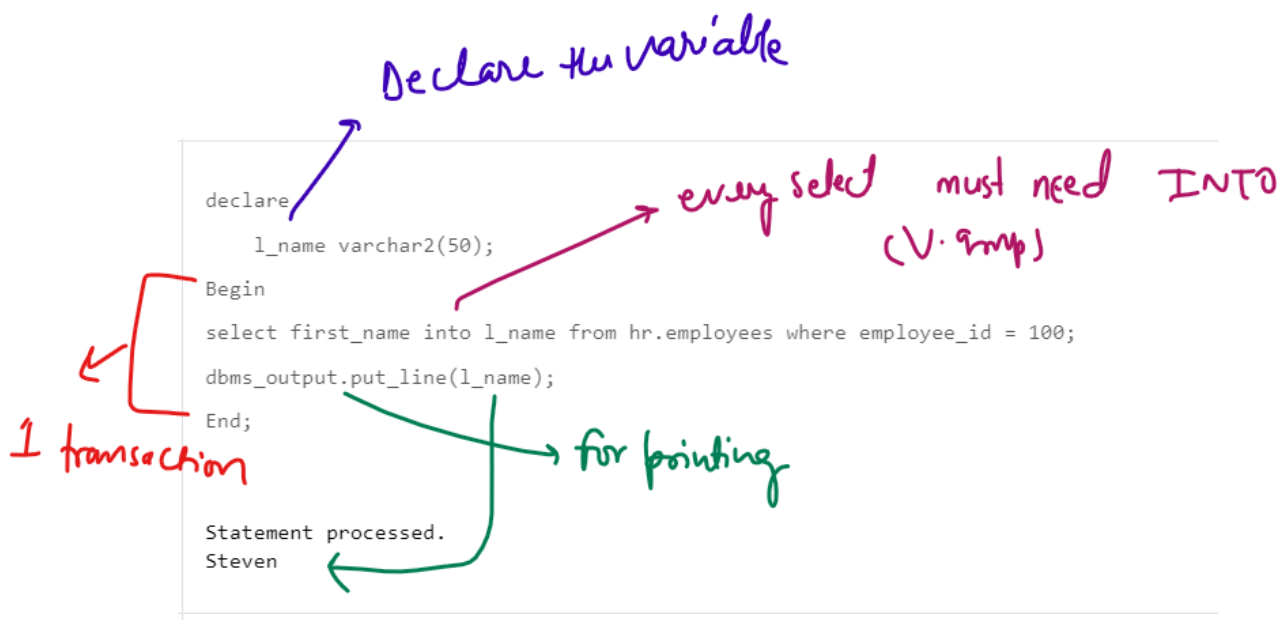
```
=====
=====
```

Select salary from hr.employees where employee_id = 100;

This above is sql. When we run this it will give us the salary of an employee 100.

Let's make pl/sql:

```
Declare
l_salary number;
Begin
  Select salary Into l_salary from hr.employees where employee_id = 100;
  Dbms_output.put_line(l_salary);
End;
```



Every select needs an INTO clause. This is the important aspect of any sql. We can not select directly like a sql statement, PL/SQL needs a placeholder, where we hold the value when selected from sql. If selecting 2 things, we need 2 variables. Its a 1:1 **and in the same sequence, in which they are selecting.**

```

declare
    l_name varchar2(50);
    l_sal number;
Begin
    select first_name, salary into l_name, l_sal from hr.employees where employee_id = 100;
    dbms_output.put_line(l_name);
End;

```

Statement processed.
Steven

2 select - 2 into

every into needs to declare

Lets Proceed to understand the same PL/SQL writing:

```

declare
l_name varchar2(20);
begin
    select region_name into l_name from hr.regions where region_id = 3;
    dbms_output.put_line(l_name);
End;

```

Statement processed.
Asia

✓ working fine for id=3

```

declare
l_name varchar2(20);
begin
    select region_name into l_name from hr.regions where region_id = 4;
    dbms_output.put_line(l_name);
End;

```

✓ fail for id=4

ORA-06502: PL/SQL: numeric or value error: character string buffer too small
 ORA-06512: at "SYS.DBMS_SQL", line 1721

See the above script. working fine for Region_id = 3 but gives error for region_id = 4. if you see the error, it is saying "character string buffer too small". for region_id = 4 name is "Middle East and Africa"

And It is total Length = 22. We declare our variable as l_name varchar2(20);

So 22 length string can not be fit into 20. So solution is always **Table_name.column_name%Type.**

By using this concept of declaration, we do not have to worry about the length or the data type. It automatically pick from table.

Declare section

tablename . columnname % type

```
Declare
l_name hr.regions.region_name%type;
Begin
select region_name into l_name from hr.regions where region_id = 4;
dbms_output.put_line(l_name);
End;
```

Statement processed.
Middle East and Africa

tablename column name

```
declare
l_name hr.employees.hire_date%Type;
begin
select hire_date into l_name from hr.employees where employee_id = 100;
dbms_output.put_line(l_name);
End;
```

Statement processed.
17-JUN-03

Assigning Values to Variables by SELECTing INTO

DECLARE

bonus NUMBER(8,2);

emp_id NUMBER(6) := 100;

BEGIN

SELECT salary * 0.10 INTO bonus FROM employees

WHERE employee_id = emp_id;

END;

See on Line #3,

emp_id NUMBER(6) := 100;

This is assignment. := , it means, right side value 100 is assigned to left side variable emp_id

Declare

a number;

b number;

c number;

Begin

a := 10;

b := 5;

c := a/b;

dbms_output.put_line(c);

End;

→ Assignment . 10 is assigned to a.
→ 5 is assigned to b

Statement processed.

2